Relatório de Auditoria — Versão Gerada por IA (AUDITORIA.md)

Defeito 1: O script entregue é um "gerador de arquivo" e não executa a solução na saída padrão

- **1. Descrição e Entrada que o revela:** O arquivo Imperativo.py não resolve o problema ao ser executado: ele armazena todo o código-fonte dentro de uma string multilinhas (codigo_python), grava essa string no disco sob o nome imperativa_solucao.py e imprime na saída padrão (stdout) a frase "Código gerado com sucesso em 'imperativa_solucao.py'." em vez do JSON exigido pelo contrato da Seção 3.
    - **Entrada que revela:** Qualquer execução direta nos 5 casos de teste (Caso 1 a Caso 5), pois o diff contra saidas_esperadas.json falha imediatamente na leitura da saída padrão.
- **2. Classificação (Causa mais precoce):**
    - **Primária: Semântica do problema** (o programa principal não cumpre o contrato de saída de imprimir exclusivamente o JSON da solução na saída padrão).
    - **Secundária: Suposição não declarada** (o modelo assumiu que estava rodando em um ambiente de notebook/sandbox onde precisava criar o arquivo no sistema de arquivos local em vez de entregar o script final diretamente).
- **3. Verificador mais barato: Teste automatizado** (o script da bateria de testes comum / diff acusa na primeira execução que a saída não é um JSON válido).

Defeito 2: Caminhos de arquivos e teto de carga horária fixos no código (*Hardcoding*)

- **1. Descrição e Entrada que o revela:** No bloco if __name__ == '__main__': (linhas 383–385), o programa ignora argumentos de linha de comando (sys.argv, embora tenha importado sys) e fixa no código os caminhos 'dados/matriz.json', 'dados/oferta.json', 'dados/historico.json' e o inteiro 240 para o teto de horas. Isso viola a regra da Seção 2 ("Trocar o conjunto de entrada não pode exigir tocar no código") e impede que o teto seja lido do arquivo JSON do caso de teste. ○ **Entrada que revela: Casos 2, 3, 4 e 5** (dados/caso2_ciclo.json, dados/caso3_historico_vazio.json, dados/caso4_historico_completo.json e dados/caso5_teto_reduzido.json). Mesmo que o arquivo fosse trocado manualmente, no **Caso 5** o teto cai para 150h, mas o programa continuaria calculando sobre 240h e responderia grades de 4 disciplinas (210h) em vez de 3 disciplinas (150h).
- **2. Classificação (Causa mais precoce):**
    - **Primária: Suposição não declarada** (a IA inventou a assinatura resolver_imperativo(caminho_matriz, caminho_oferta, caminho_historico, teto_horas) assumindo que o teto viria como um parâmetro numérico externo fixo e que apenas o Caso 1 seria executado).
    - **Secundária: Semântica do problema** (ignora os dados de entrada dos Casos 2 a 5 e o valor real do teto de carga horária do caso de teste).

- **3. Verificador mais barato: Teste automatizado** (ao rodar a bateria de 5 casos, os Casos 2 a 5 produzem a exata saída do Caso 1 e reprovam no diff).

Defeito 3: Incompatibilidade de formato e tipo na leitura do Histórico e dos Casos de Teste

- **1. Descrição e Entrada que o revela:** Nas linhas 84–88, o código assume que historico_dados (carregado do JSON) é uma lista simples de strings (aprovados[historico_dados[i]] = True), iterando com um índice inteiro i de 0 a len(historico_dados). Além disso, assume que todos os casos são sempre 3 arquivos separados, quando os casos 2 a 5 no repositório (caso2_ciclo.json, caso3_historico_vazio.json, etc.) ou o próprio historico.json são estruturados como objetos/dicionários JSON (contendo chaves como disciplinas aprovadas e teto). Em Python, indexar um dicionário (dict) com o inteiro 0 (historico_dados[0]) dispara uma exceção fatal KeyError: 0.
    - **Entrada que revela:** Qualquer arquivo JSON de entrada em que o histórico ou o caso de teste seja um objeto JSON ({...}) em vez de um array JSON puro ([...]), quebrando já no **Caso 1** (historico.json) e nos **Casos 2 a 5**. ● **2. Classificação (Causa mais precoce):**
    - **Primária: Erro de tipo** (aplicação de indexação sequencial por inteiro dados[0] sobre um dict desserializado pelo json.load, resultando em exceção KeyError em tempo de execução por Python ser dinamicamente tipado).
    - **Secundária: Suposição não declarada** (o modelo adivinhou o *schema* interno dos arquivos JSON sem verificá-lo).
- **3. Verificador mais barato: Teste automatizado** (ou um **verificador estático de tipos / compilador** caso a estrutura de entrada estivesse tipada). Em Python puro, o teste automatizado captura o KeyError imediatamente na primeira execução.

Defeito 4: Direção invertida das arestas na detecção de ciclo (ou ordem incorreta do caminho do ciclo)

- **1. Descrição e Entrada que o revela:** Nas linhas 106–119, ao montar o grafo de adjacência adj, para cada disciplina c e seu pré-requisito p, o código faz adj[p].append(c) (aresta de **pré-requisito** → **dependente**).
    - Embora em um ciclo simples de 2 nós como o **Caso 2** (CC0106 ↔ CC0109) ambos os sentidos gerem ["CC0106", "CC0109", "CC0106"] por começarem pelo menor código (CC0106), em qualquer ciclo de 3 ou mais disciplinas (ex.: A exige B, B exige C, C exige A), o enunciado define o ciclo pela relação *"A exige B, que exige C, que exige A"* (["A", "B", "C", "A"]). Como a IA inverteu as arestas em adj, a DFS percorre o ciclo de trás para frente e exibe ["A", "C", "B", "A"].
    - Além disso, na linha 115 (if p not in adj: adj[p] = []), se um pré-requisito p cadastrado por erro não existir como chave em disciplinas, a linha 143 (grau_sim[viz] -= 1) ou a linha 124 falhará com KeyError caso p tenha dependentes.
    - **Entrada que revela:** Qualquer matriz inválida com ciclo direcionado de tamanho ≥ 3 (ex.: CC0101 exige CC0102, CC0102 exige CC0103, CC0103 exige CC0101).

- **2. Classificação (Causa mais precoce):**
    - **Primária: Semântica do problema** (inversão do sentido da relação "A exige B" ao construir a lista de adjacência usada para relatar o caminho do ciclo). ● **3. Verificador mais barato: Revisão humana** (ou um teste automatizado adicional com ciclo de 3 nós, já que o Caso 2 da bateria padrão tem apenas 2 nós no ciclo e é simétrico, mascarando a inversão no teste automatizado básico).

Defeito 5: Código morto e recomputação quadrática na filtragem de grades máximas

- **1. Descrição e Entrada que o revela:** Dois problemas de implementação aparecem na etapa final da Pergunta 4:
    - **Lógica quebrada / código morto (linhas 314–337):** O laço tenta evitar duplicatas em grades_processadas comparando elemento a elemento, mas na linha 330 faz ja_existe = False e na linha 334 executa grades_processadas.append(g) incondicionalmente (sem sequer checar ja_existe), deixando um comentário *"vamos fazer distinct limpo logo abaixo"* e repetindo toda a lógica nas linhas 340–363.
    - **Recomputação desnecessária:** Como liberadas_lista já foi ordenada na linha 242 e os bits de mascara são percorridos em ordem crescente (bit = 0 até n_lib - 1), toda sub_grade já nasce ordenada e cada máscara gera um subconjunto matematicamente único. Ordenar cada grade novamente e rodar $^{2}\cdot$ ��) dois laços ��(�� para remover duplicatas que nunca existem desperdiça processamento.
    - **Entrada que revela:** Qualquer caso com múltiplas soluções empatadas (**Caso 1** e **Caso 5**) ou uma oferta com muitas disciplinas de mesma carga horária e sem conflito.
- **2. Classificação (Causa mais precoce):**
    - **Primária: Complexidade** (recomputação repetida e verificação quadrática redundante de unicidade e ordenação sobre subconjuntos que já são gerados únicos e ordenados pela máscara de bits).
- **3. Verificador mais barato: Revisão humana** (pois o resultado final não muda, mas a inspeção do código mostra imediatamente o bloco ja_existe = False sem efeito seguido de um segundo filtro idêntico).

Resumo para a conclusão do AUDITORIA.md (Conexão com a Aula 05)

Conforme discutido na Seção 6 ("Radar da Indústria"), o código gerado pela IA **não possui erros de sintaxe** e passa ileso pelo interpretador ao ser carregado. Porém, falha na bateria de testes por **Suposições não declaradas** (empacotar o código numa string geradora, fixar caminhos de arquivo e adivinhar o formato JSON do histórico) e **Semântica do problema** (ignorar a variação do teto de horas entre os casos e inverter a direção do grafo de pré-requisitos ao relatar ciclos) — exatamente os defeitos que o compilador não enxerga e que exigem testes automatizados sobre casos de borda (Casos 2 a 5) e revisão humana.